



UiO : University of Oslo

A Comparative Study for Single Cell Viability Quantification Using
Machine Learning Methods
Autumn 2025

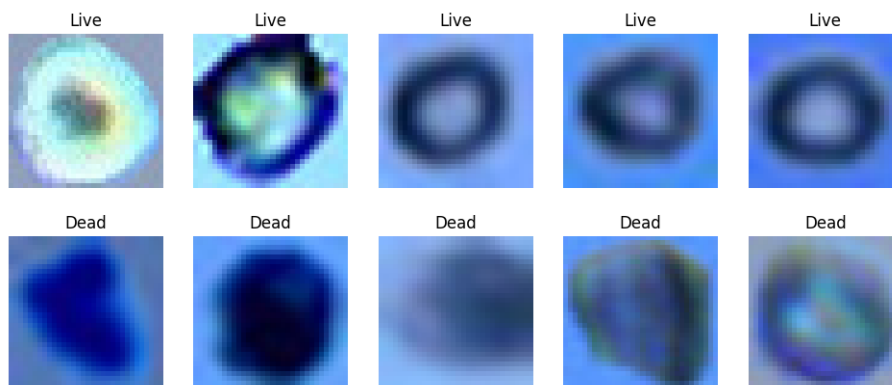
Project 3 on Machine Learning
(Deadline: December 18 (midnight), 2025)

Author:

Samson ST#: 555652 & Theodor
ST#: 0501579 & Bartosz ST#:
703557

Supervisor: Prof. Morten

Hjorth-Jensen



December 17, 2025

Contents

1	Introduction	3
2	Methods and Implementation	3
2.1	Data and split	4
2.1.1	Dataset	4
2.1.2	Data split usage	5
2.2	Learning Models	5
2.2.1	Unsupervised learning	5
2.2.2	Supervised learning	6
2.3	Preprocessing and representations	6
2.3.1	Raw pixel representation	6
2.3.2	Grayscale pixel representation	7
2.3.3	Engineered feature representation	7
2.4	Applied Dimensionality Reduction	7
2.4.1	PCA on pixel representation	7
2.4.2	PCA on engineered features	8
2.5	Applied clustering with k-means	8
2.6	Applied supervised classification	9
2.6.1	MLP classifiers	9
2.6.2	CNN classifier	9
2.7	Metrics and evaluation	10
2.8	Reproducibility and verification	10
3	Use of AI tools	10
4	Experimental Results and Analysis	11
4.1	Dataset inspection and preprocessing	11
4.2	PCA analysis and representation quality	11
4.2.1	PCA explained variance across representations	11
4.2.2	3D PCA visualization	12
4.3	Unsupervised learning results	12
4.3.1	PCA dimensionality sweep for clustering	12
4.3.2	PCA + k-means on raw pixels	13
4.3.3	PCA + k-means on engineered features	13
4.3.4	Analysis of engineered features	13
4.4	Supervised MLP results	14
4.5	CNN training dynamics and performance	15
4.6	Comparative evaluation across methods	15
4.6.1	Performance comparison	15
4.6.2	Error analysis	15
4.6.3	Computational cost comparison	17
4.6.4	Final test set evaluation	18
5	Discussion	18
6	Conclusions	19

List of Figures

1	Example grid of random live/dead images from the dataset	4
2	Architecture of the convolutional neural network used for supervised classification. The network takes $32 \times 32 \times 3$ RGB images as input and applies two convolutional blocks with ReLU activations and max pooling, followed by fully connected layers that output class logits.	9
3	Explained variance ratio of PCA components computed on the training set.	11
4	Three-dimensional PCA projections of pixel-based (left) and feature-based (right) representations.	12
5	Development-set F1-score heatmap for MLP hyperparameter selection. The parameter α denotes the scaling factor of the ℓ_2 regularization term.	14
6	CNN training dynamics. Left: training and development loss curves. Right: accuracy as a function of learning rate and batch size on the development set.	15
7	Comparison of model performance on the development set across multiple metrics. .	16
8	Comparison of computational cost across models.	17
9	Comparison of computational cost across models.	18

Abstract

Distinguishing live and dead cells in microscopy images is commonly approached with supervised learning, but labels can be costly to obtain. We therefore examine how far unsupervised learning can go, and how it compares to supervised models when labels are available.

We evaluate PCA+k-means clustering and compare it to supervised baselines (MLP and CNN) on the same train/dev/test split. Clustering on PCA-reduced raw pixels performs poorly (dev $F_1 \approx 0.35$), while adding simple engineered features substantially improves unsupervised performance ($F_1 \approx 0.68$). Supervised models perform better: an MLP reaches $F_1 \approx 0.77$ on pixel-PCA and $F_1 \approx 0.91$ on feature-PCA, and a compact CNN achieves near-perfect results ($F_1 = 1.00$, test $F_1 \approx 0.99$).

Overall, the results show that unsupervised performance is primarily representation-limited: meaningful structure emerges once nuisance variation is reduced and morphology-relevant cues are emphasized. We find that with sufficient labels, end-to-end CNNs provide the highest accuracy and best generalization.

1 Introduction

Accurately distinguishing live from dead cells in microscopy images is a fundamental task in biological research. Reliable assessment of cell viability is essential for applications ranging from drug screening to toxicity studies. In recent years, machine learning methods particularly supervised deep learning have achieved strong performance in image-based cell classification tasks. However, these approaches typically rely on large, carefully annotated datasets, where labeling requires expert knowledge and substantial manual effort. In many practical settings, such labeled data are scarce, expensive, or unavailable.

This challenge motivates the exploration of unsupervised learning methods, which aim to discover structure in data without relying on ground-truth labels. While unsupervised techniques cannot be optimized directly for a specific classification objective, they offer the potential to identify meaningful biological patterns and to serve as viable alternatives when annotation resources are limited. A key open question is therefore how effectively unsupervised methods can separate biologically relevant cell states, and how their performance compares to supervised models when labels are available.

In this project, we investigate the extent to which unsupervised clustering can distinguish live and dead cells in microscopy images. We evaluate clustering pipelines based on Principal Component Analysis (PCA) and k-means, applied both to raw pixel representations and to hand-engineered, biologically motivated features. To contextualize the performance of these unsupervised approaches, we compare them directly against supervised models trained on the same data, including a multi-layer perceptron (MLP) and a convolutional neural network (CNN). By systematically comparing representations, learning paradigms, and evaluation metrics under a fixed experimental protocol, this study aims to clarify both the limitations and the potential of unsupervised learning for cell viability classification.

2 Methods and Implementation

For the following material, we assume familiarity with basic supervised and unsupervised learning concepts (PCA, k-means, and neural networks); we therefore do not repeat the underlying mathematical derivations here and instead refer the reader to the course notes and standard references for theory, focusing on the concrete experimental pipeline and evaluation protocol used in this project. This project compares five pipelines for distinguishing live vs dead cells in microscopy images. There are two main goals:

1. Test how far unsupervised learning can go when labels are not available.
2. Evaluate that against supervised learning when labels are available.

Therefore, we evaluate:

- PCA + k-means on raw pixels (unsupervised, minimal assumptions)
- PCA + k-means on engineered features (unsupervised, hand-engineered representation)
- MLP on pixel-PCA (supervised, low-dimensional embeddings)
- MLP on feature-PCA (supervised, compact and interpretable representation)
- CNN on raw images (supervised end-to-end model)

All experiments follow the same protocol described below 2.8

2.1 Data and split

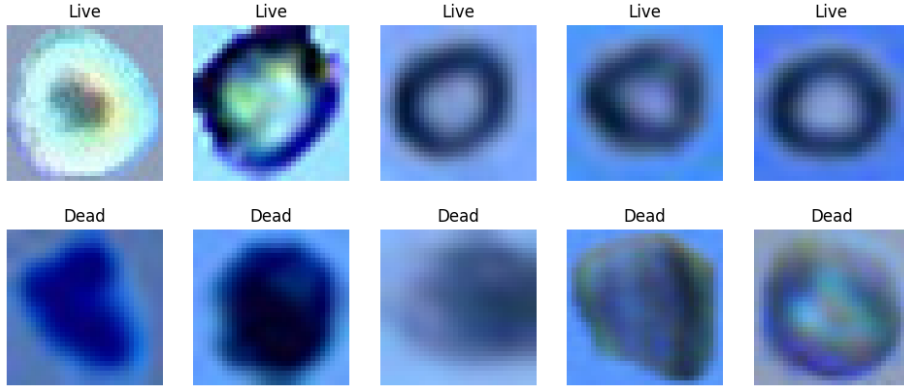


Figure 1: Example grid of random live/dead images from the dataset

2.1.1 Dataset

This study uses a publicly available dataset [2] of trypan-blue stained cells. Our dataset was synthesized using the original dataset. The original dataset contained images with many cells and bounding box labels. The single cell bounding boxes were used to crop each cell into unique images. Then these cropped images were resampled to size $32 \times 32 \times 3$ with three color channels: Red, Blue, and Green. The label for each single cell is a binary classification as follows:

- 0 = live cell
- 1 = dead cell

The dataset is class-imbalanced - live cells are a majority group.

Accuracy 2.7 of a trained classifier alone can be misleading in this setting because it is dominated by the majority class and does not distinguish between different types of errors. Given the class

imbalance in the dataset, a classifier that predominantly predicts the majority class could achieve high accuracy while performing poorly on the minority class. For this reason, we emphasize precision, recall, and the F1 score 2.7, which explicitly capture performance on the minority class and the trade-off between false positives and false negatives.

As a basic sanity check, we visualize a small random subset of images from each class after loading (Fig. 1). This confirms that the label mapping is correct, the images have the expected size and number of channels, and that there are no obvious loading issues. We also report the number of samples per class in each split to verify that the provided split structure is loaded correctly and to quantify the degree of class imbalance (live/dead ratio ≈ 2.72 , corresponding to $\approx 73\%$ live samples).

2.1.2 Data split usage

We follow a fixed protocol (Sec:2.8) throughout all experiments with an 80/10/10 split:

- **Training set (80%)** used to fit preprocessing steps and to train all models, consisting of 416 dead and 1131 live cells.
- **Development set (10%)** used for model selection and hyperparameter tuning (e.g. feature thresholds and radii, PCA dimensionality, CNN learning rate), consisting of 52 dead and 141 live cells.
- **Test set (10%)** evaluated once for the final selected model to estimate generalization performance, consisting of 52 dead and 142 live cells.

We follow the fixed protocol described below to avoid optimistic bias in the reported performance.

2.2 Learning Models

We explore both unsupervised and supervised learning methods. In unsupervised learning, models are fit without access to ground-truth labels; evaluation is performed post-hoc by comparing predicted clusters to the true classes. In supervised learning, labels are provided during training and model parameters are optimized to directly predict the target class. Our method choices follow the course lecture material on dimensionality reduction, clustering, and neural networks[6], with particular emphasis on "Dimensionality Reduction" and "Deep Learning Methods" sections of the lecture's jupyter notebook.

2.2.1 Unsupervised learning

Unsupervised learning is used to test classification performance without label information. In general, unsupervised methods rely on assumptions about the geometry of the data (e.g., that samples form separable groups in a suitable representation).

Principal Component Analysis (PCA). Principal Component Analysis (PCA) is a linear dimensionality reduction technique that fits a low-dimensional affine subspace S (of dimension $d \ll D$ where D is the original dimensionality) to the data by rotating the coordinate system so that the new orthogonal axes, the principal components, are ordered by decreasing explained variance[6, 14]. The first components capture the dominant directions of variation in the dataset, enabling compression while preserving as much variance as possible under a linear projection. PCA can be used as a preprocessing step to reduce dimensionality and to mitigate redundancy due to correlated input dimensions.

K-means clustering. K-means (also known as Lloyd’s algorithm) is a simple and widely used centroid-based method for cluster analysis, i.e., grouping a data set into k distinct categories based on a chosen similarity/dissimilarity measure, most commonly a distance metric such as the Euclidean distance. It partitions the samples by minimizing the within-cluster sum of squared Euclidean distances to the cluster centroids [6]. The algorithm alternates between assigning each sample to its nearest centroid and updating each centroid as the mean of the samples assigned to it. Because its objective relies on squared Euclidean distances, k-means works best when clusters are roughly spherical and similarly scaled in the chosen representation.

2.2.2 Supervised learning

Supervised learning uses the provided labels to directly optimize classification performance. Models are trained by minimizing a loss function that penalizes incorrect predictions, typically using gradient-based optimization.

Multilayer Perceptron (MLP). The multilayer perceptron is a feed-forward neural network composed of fully connected layers with nonlinear activation functions [6, 12]. By stacking linear transformations with nonlinearities, MLPs can approximate complex decision boundaries in feature space. Training typically minimizes a cross-entropy loss for classification via backpropagation and gradient-based optimization.

Convolutional Neural Network (CNN). Convolutional neural networks are designed to exploit spatial structure in images through local receptive fields, shared weights, and pooling operations [6, 3, 8]. Convolution layers learn spatially localized patterns (e.g., edges, textures) that can be composed hierarchically into higher-level features. Compared to fully connected networks on flattened pixels, CNNs incorporate an inductive bias for translation-locality and spatial coherence, which is often advantageous in image classification tasks.

2.3 Preprocessing and representations

To compare unsupervised and supervised approaches, we evaluate multiple input representations derived from the same images. All preprocessing steps follow the same protocol (Sec:2.8). For models that operate on vector inputs (PCA, k-means, MLP), images are converted into fixed-length vectors. For CNNs, the original image structure is preserved.

In our experiments, scaling depends on the chosen representation. Engineered features are standardized (zero mean, unit variance) using training-set statistics before PCA, and the resulting PCA embeddings are then used for downstream models. For the pixel-based MLP, we do not manually rescale raw RGB intensities; instead, scaling is handled by a `StandardScaler` inside the MLP pipeline. In the grayscale PCA sweep, intensities are explicitly scaled to $[0, 1]$ prior to PCA [6, 13, 14].

2.3.1 Raw pixel representation

The baseline representation is the raw RGB image crop of shape $32 \times 32 \times 3$. This representation is used in two ways:

- **Vector form:** For PCA, k-means, and the pixel-based MLP input, each image is flattened into a vector of length $p = 32 \cdot 32 \cdot 3 = 3072$.

- **Tensor form:** For the CNN, images are kept in their original spatial structure and provided as a $3 \times 32 \times 32$ tensor.

This raw representation makes minimal assumptions about cell morphology, but it may include background variation and illumination differences that contribute strongly to pixel-level variance.

2.3.2 Grayscale pixel representation

As an additional pixel-based representation, we also consider grayscale images computed from RGB using a standard luminosity conversion, where the grayscale intensity is formed as a weighted sum of the red, green, and blue channels[1]. The resulting image has shape 32×32 and is flattened into a vector of length $p = 1024$.

For the grayscale PCA variance sweep, grayscale intensities are explicitly scaled to $[0, 1]$ prior to fitting PCA. This representation is used only for exploratory comparisons in the unsupervised PCA + k-means setting.

2.3.3 Engineered feature representation

In addition to raw pixel inputs, one can represent each image by a compact set of engineered (hand-crafted) features. The purpose of feature engineering is to summarize task-relevant image properties into a low-dimensional vector that is easier to model than raw pixels. In microscopy settings, engineered features are often motivated by simple morphological hypotheses, such as differences in intensity distribution, cell interior structure, and boundary sharpness[13].

2.4 Applied Dimensionality Reduction

We apply PCA as a practical dimensionality reduction step in several pipelines. We use PCA as described in the lecture notes[6] and summarize how it is applied in our experiments. In all cases, PCA follows the same fitting protocol (Sec:2.8).

In this project, PCA serves three roles:

- **Compression:** reduce input dimensionality before clustering and before training MLP classifiers.
- **Noise filtering:** discard low-variance directions that may correspond to background variation, illumination changes, or pixel-level noise.
- **Visualization:** enable low-dimensional embeddings (e.g., 3D) for qualitative inspection of separability.

2.4.1 PCA on pixel representation

For the pixel-based pipelines, PCA is fitted on flattened image vectors (length 3072 for RGB). The resulting PCA embeddings provide a compact representation that is used both for visualization and as input to downstream learning algorithms. The number of kept components m is explored using a variance sweep (cumulative explained variance) and, for clustering experiments, by evaluating development-set performance for different values of m .

2.4.2 PCA on engineered features

To provide a more compact and task-relevant representation, we visually inspect the dataset which allows us to design a set of hand-crafted features intended to capture structural differences between live and dead cells. Features are extracted from grayscale images using the function `extract_cell_features()` (see accompanying code module). The feature design targets three intuitive image properties:

- overall intensity and contrast,
- center-versus-background structure,
- edge/texture cues that relate to boundary strength and image sharpness.

In total, we compute a 12-dimensional feature vector per image. Table 1 summarizes the feature set and its intended interpretation.

Before applying PCA, the engineered feature vectors are standardized to zero mean and unit variance using training-set statistics. Since this representation is already low-dimensional, PCA is mainly used to decorrelate and compact the feature space, producing a small set of orthogonal components for downstream modeling. Feature extraction parameters (e.g., center/ring radii and the dark-pixel threshold) and the final PCA dimensionality and feature parameters are selected using development-set performance in the feature-based pipeline.

Table 1: Engineered features extracted per image (implemented in `extract_cell_features()`).

Feature	Category	Captures
<code>gmean</code>	global	overall brightness
<code>gstd</code>	global	global contrast
<code>cmean</code>	center	core brightness
<code>rmean</code>	ring	surrounding brightness
<code>c_minus_r</code>	contrast	center–ring difference
<code>c_over_r</code>	contrast	center–ring ratio
<code>center_p90</code>	peak	central bright spot statistic
<code>peak_over_ring</code>	peak	peak vs. ring contrast
<code>radial_std</code>	structure	radial profile variation
<code>edge_energy</code>	edges	boundary/edge strength
<code>lap_var</code>	texture	sharpness
<code>dark_frac</code>	darkness	proportion of dark pixels

2.5 Applied clustering with k-means

We use k-means clustering as the unsupervised method for assigning samples into two groups corresponding to the two classes. Throughout this project we fix $k = 2$.

In our experiments, k-means is applied in a PCA-reduced space (either pixel-PCA or feature-PCA). This reduces dimensionality and helps mitigate redundancy in the inputs before clustering. Since cluster indices are arbitrary (cluster 0 vs. cluster 1 has no inherent meaning), we align predicted cluster labels with the classes using a label flip on the development set. Concretely, we compare the F1 score of the original assignment and the flipped assignment, and retain the labeling that maximizes F1 (implemented as `best_flip` in the accompanying code module). This alignment is used only for evaluation and does not change our clustering solution.

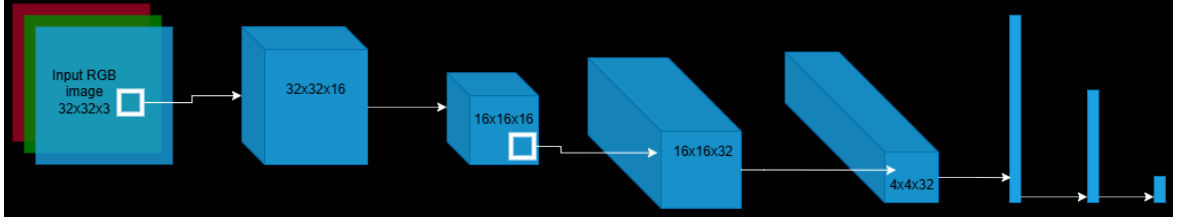


Figure 2: Architecture of the convolutional neural network used for supervised classification. The network takes $32 \times 32 \times 3$ RGB images as input and applies two convolutional blocks with ReLU activations and max pooling, followed by fully connected layers that output class logits.

2.6 Applied supervised classification

In the supervised setting, ground-truth labels are used to train models that directly predict the class label (live/dead). We evaluate two supervised classifiers: an MLP and a CNN trained end-to-end on raw images (Sec:2.2). In all cases, hyperparameters are selected following a standard train–development–test protocol (Sec:2.8) [3, 5].

2.6.1 MLP classifiers

We use a multilayer perceptron (MLP) classifier as a supervised model operating on vector inputs. Two input representations are considered:

- **Pixel-PCA input:** PCA-reduced embeddings computed from flattened RGB images.
- **Feature-PCA input:** PCA-reduced embeddings computed from engineered cell features.

In both cases, dimensionality reduction with PCA is performed using our reproducibility protocol (Sec:2.8). The MLP training follows the theoretical setup (Sec: 2.2) [3]. For pixel-based inputs, feature scaling is applied via a `StandardScaler` as part of the training pipeline. Our model uses two hidden layers. Model hyperparameters, including hidden-layer width (i.e. number of nodes per hidden layer) and ℓ_2 regularization strength, are selected based on development-set performance. The implementation uses the `MLPClassifier` provided by scikit-learn [11].

2.6.2 CNN classifier

To directly exploit the spatial structure of the images, we implement a compact convolutional neural network (CNN) trained end-to-end on raw $32 \times 32 \times 3$ RGB images. The architecture consists of two convolutional blocks followed by fully connected layers 2. Specifically, the first block applies a 3×3 convolution with 16 output channels, ReLU activation, and 2×2 max pooling, reducing the spatial resolution to 16×16 . The second block applies a 3×3 convolution with 32 channels, ReLU activation, and 4×4 max pooling, yielding a $4 \times 4 \times 32$ feature map. The resulting representation is flattened and passed through a fully connected layer with 64 hidden units and ReLU activation, followed by a linear output layer producing class logits.

The network training follows our theoretical setup (Sec: 2.2) and is optimized with the Adam optimizer [7]. We use the preprocessing described above (Sec: 2.3). Hyperparameters tuning such as learning rate and batch size as well as the final reported performance are obtained following our reproducibility protocol below (Sec:2.8)[3].

2.7 Metrics and evaluation

Since the dataset is class imbalanced, we do not rely on accuracy alone. Instead, we report multiple complementary metrics[6, 10].

Accuracy. Accuracy measures the fraction of correctly classified samples,

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN},$$

where TP , TN , FP , and FN denote true positives, true negatives, false positives, and false negatives, respectively.

F1 score. To account for class imbalance, we emphasize the F1 score, defined as the harmonic mean of precision and recall,

$$F_1 = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}, \quad \text{precision} = \frac{TP}{TP + FP}, \quad \text{recall} = \frac{TP}{TP + FN}.$$

F1 is informative when the minority class is important and when a model may trade precision against recall.

ROC AUC. We also report ROC AUC as a threshold-independent measure of separability between classes[10]. In the binary setting, ROC AUC summarizes the model’s ability to rank positive samples above negative samples across all possible decision thresholds.

Confusion matrices. For qualitative error analysis, we report confusion matrices that show the distribution of predictions across true classes. This makes it possible to identify whether errors are dominated by false positives or false negatives, which is particularly relevant in imbalanced datasets.

2.8 Reproducibility and verification

All experiments are implemented in Python using standard scientific computing and machine learning libraries[10, 4]. To ensure reproducibility, we fix random seeds for NumPy, Python, and (when applicable) PyTorch, and we use deterministic settings where supported.

To prevent information leakage, preprocessing steps (standardization, PCA fitting) are fitted on the training set only and applied unchanged to the development and test sets. Model selection and hyperparameter tuning are performed exclusively on the development set, and the test set is evaluated once for the final selected model.

We additionally record wall-clock runtimes for key steps (e.g., PCA fitting, clustering, and model training) to compare computational cost across methods. All timing values are hardware-dependent and should be interpreted as relative comparisons rather than absolute benchmarks.

3 Use of AI tools

We used ChatGPT[9] to assist with drafting text and troubleshooting coding issues. All generated suggestions were reviewed, verified, and edited by us to ensure correctness and relevance to the project.

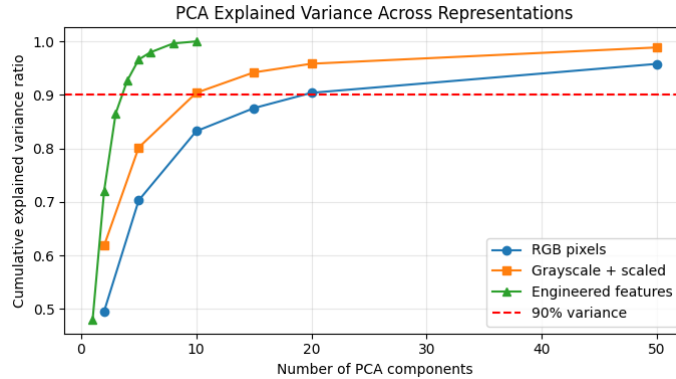


Figure 3: Explained variance ratio of PCA components computed on the training set.

4 Experimental Results and Analysis

This section presents the experimental results for all evaluated methods, including unsupervised PCA-KMeans baselines and supervised MLP and CNN classifiers. Performance is assessed using accuracy, F1-score, ROC AUC, confusion matrices, and computational cost.

4.1 Dataset inspection and preprocessing

As a basic sanity check, we visualize a small random subset of images from each class after loading (Fig. 1). This confirms that the label mapping is correct, the images have the expected spatial resolution and color channels, and that no obvious data-loading issues are present. We also verify the number of samples per class in each split, revealing a moderate class imbalance between live and dead cells.

4.2 PCA analysis and representation quality

4.2.1 PCA explained variance across representations

Using the PCA protocol described in Sec. 2.4, we compute the cumulative explained variance ratio (EVR) for three representations: flattened RGB pixels, grayscale pixels, and engineered features. Fig. 3 summarizes how quickly each representation can be compressed under a linear projection, with the dashed line marking the 90% threshold.

The raw RGB pixel representation requires the largest number of components to reach 90% EVR - approximately $m \approx 20$ in our sweep - indicating that variance is distributed across many directions in the original 3072-dimensional space. The grayscale representation concentrates variance into fewer components (roughly $m \approx 10$ to reach 90%), consistent with reduced redundancy after collapsing color channels. Finally, the engineered feature representation is the most compact: since the input is already low-dimensional and structured (Sec. 2.4), only a few components are needed to exceed 90% EVR (around $m \approx 4$).

These variance profiles quantify the degree of linear compressibility of each representation. As emphasized in Sec. 2.4, explained variance alone does not guarantee class separability; it only measures how well the data can be approximated in a lower-dimensional subspace.

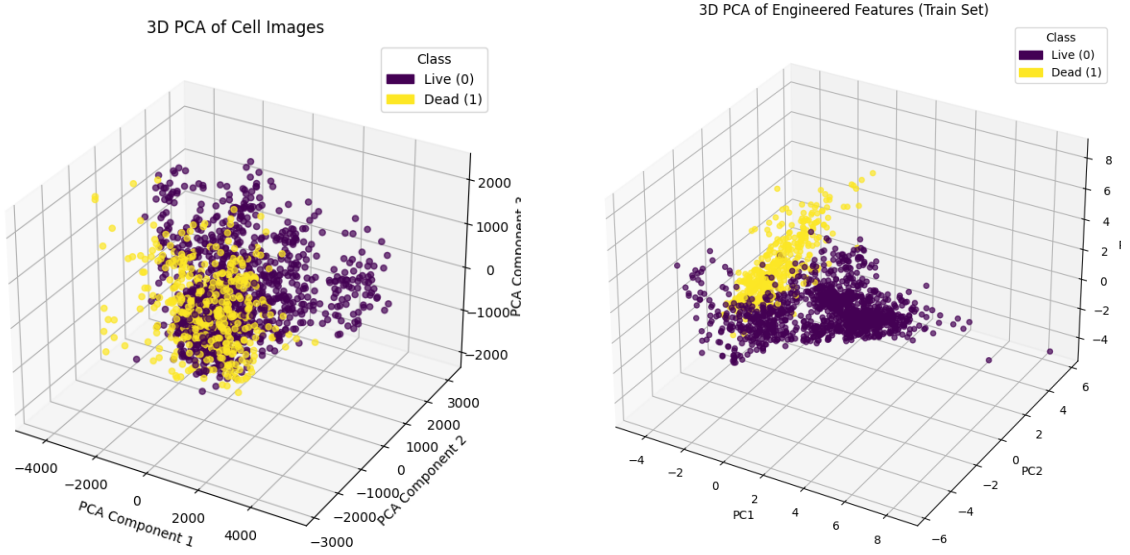


Figure 4: Three-dimensional PCA projections of pixel-based (left) and feature-based (right) representations.

4.2.2 3D PCA visualization

Following the dimensionality reduction procedure described in Sec. 2.4, we project the training data onto the first three principal components for two representations: the pixel-based PCA embedding and the engineered-feature PCA embedding. Fig. 4 shows the resulting 3D scatter plots, colored by the true class labels.

In the **pixel-based projection** (Fig. 4, left), the two classes show substantial overlap in the first three principal components. This indicates that the dominant directions of variance in raw pixels are not aligned with the live/dead distinction, and that class-relevant structure is not captured by this low-dimensional linear projection.

In contrast, the **feature-based projection** (Fig. 4, right) shows a strikingly clearer separation between the two classes. This suggests that the engineered features emphasize image properties that are more directly related to cell state, so that the main variance directions in feature space are more informative for discrimination. This qualitative difference is consistent with the improved downstream performance observed for feature-based methods in later sections.

4.3 Unsupervised learning results

4.3.1 PCA dimensionality sweep for clustering

To evaluate how the choice of PCA dimensionality affects unsupervised clustering, we sweep the number of retained principal components m and measure development-set performance after applying k-means in the PCA-reduced space (Sec. 2.4). In this sweep we use the grayscale representation described in Sec. 2.3, and we fix $k = 2$ for k-means (Sec. 2.4). Performance is reported after aligning cluster indices via the post-hoc label flip described in Sec. 2.4.

Across the tested values $m \in \{2, 5, 10, 15, 20, 50\}$, clustering performance remains limited and does not improve with dimensionality. The best F1 scores occur for small to moderate m (e.g., $F_1 \approx 0.42$

for $m = 2$ and $m = 5$), while larger m does not provide consistent gains. Notably, $m = 10$ yields a higher accuracy but a substantially lower F1 score, indicating that the clustering solution is biased toward the majority class under class imbalance (Sec. 2.7). This illustrates that accuracy alone can be misleading for evaluating unsupervised partitions in this dataset.

Overall, the sweep suggests that even after collapsing RGB to grayscale and applying consistent scaling (Sec. 2.3), the dominant variance directions in pixel space do not reliably produce a cluster structure aligned with the live/dead labels. We therefore retain earlier pixel-based PCA-k-means result as a baseline and proceed to engineered features, where the representation is explicitly designed to emphasize biologically meaningful image structure.

4.3.2 PCA + k-means on raw pixels

As a minimal-assumption unsupervised baseline, we apply k-means clustering ($k = 2$) to PCA-reduced raw RGB pixel vectors (Secs. 2.3–2.4). Predicted clusters are aligned with the semantic classes using our label flip (Sec. 2.4). Using this setup, the raw pixel pipeline achieves only modest performance on the development set (approximately $F_1 \approx 0.35$ and accuracy ≈ 0.33).

These results indicate that clustering structure in the raw pixel space is not well aligned with the live/dead distinction, even after linear compression with PCA. A likely explanation is that raw pixels contain substantial nuisance variation (e.g., illumination differences and background intensity patterns) that dominates the variance directions captured by PCA (Sec. 2.3). In later sections we therefore compare this baseline against feature-based clustering and supervised models, where the representation and/or objective is better matched to the classification task. Detailed confusion matrices and metric comparisons are deferred to Sec. 4.6.1.

4.3.3 PCA + k-means on engineered features

Using the engineered feature representation and the PCA-k-means protocol from Secs. 2.3–2.4, we obtain a clear improvement over the raw-pixel baseline. On the development set, the feature-based clustering reaches approximately $\text{Acc} \approx 0.75$ and $F_1 \approx 0.68$.

More importantly, the ROC AUC increases to ≈ 0.82 , approaching the level achieved by some supervised methods. This suggests that the engineered representation produces a substantially more class-aligned geometry than raw pixels, even in an unsupervised setting. The remaining errors are dominated by false positives (live cells assigned to the dead cluster), while false negatives are rare (only one dead cell predicted as live in our development set). Since the representation is hand-designed, this error pattern also indicates a concrete avenue for improvement: refining the feature set or its parameters could plausibly reduce these remaining false positives. Detailed confusion matrices and cross-method comparisons are deferred to Sec. 4.6.1.

4.3.4 Analysis of engineered features

To better understand why engineered features improve clustering performance, we analyze the variability of individual features and the structure of the resulting PCA space. A variance ranking over the (unscaled) feature dimensions shows that ratio- and peak-based descriptors dominate the raw spread, with `peak_over_ring` and `c_over_r` exhibiting the largest variances, followed by `center_p90` and `cmean`. In contrast, some texture/derivative-based features (notably `lap_var`) show very small variance in the raw scale, motivating the standardization step used in the feature pipeline (Sec. 2.3). After standardization, PCA of the engineered features yields a strongly concentrated variance profile: the first three principal components explain approximately 48.2%, 23.8%, and 14.6% of the variance, respectively (cumulative $\approx 86.6\%$). This confirms that the 12-dimensional feature space

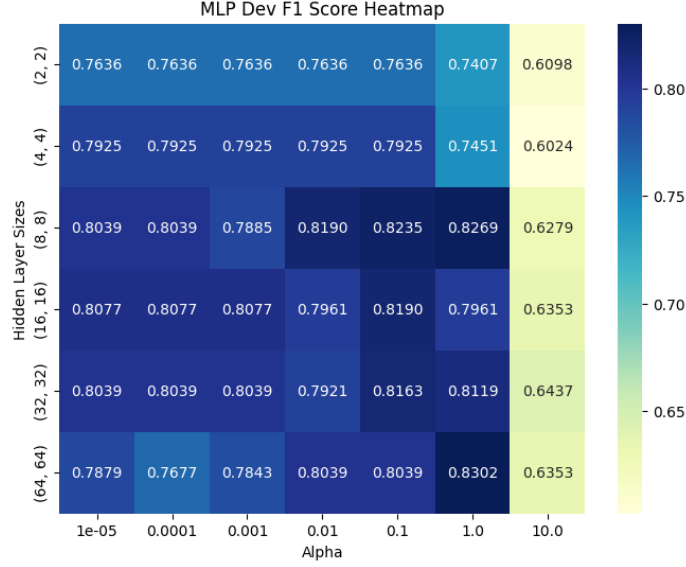


Figure 5: Development-set F1-score heatmap for MLP hyperparameter selection. The parameter α denotes the scaling factor of the ℓ_2 regularization term.

can be represented accurately with only a few orthogonal directions, consistent with the compactness observed in the explained-variance comparison (Sec. 2.4).

Inspecting PCA loadings provides an interpretable view of what each component captures. The first principal component is driven primarily by center–ring contrast and structural cues (`c_minus_r`, `edge_energy`, `radial_std`), suggesting that global boundary strength and radial organization are major axes of variation in the dataset. The second component is dominated by background/overall brightness terms (`rmean`, `gmean`) with an opposing contribution from `dark_frac`, consistent with changes in illumination and dark-pixel spread. The third component emphasizes global contrast/sharpness proxies (`lap_var`, `gstd`) together with a contribution from the center-to-ring ratio (`c_over_r`). Overall, these loadings indicate that the feature representation organizes the data along interpretable directions related to contrast, radial structure, and edge/texture factors that are likely linked to morphological differences between live and dead cells and help explain the improved separability observed in the feature-based PCA and clustering results.

4.4 Supervised MLP results

Supervised MLP classifiers trained on PCA-reduced inputs substantially outperform the unsupervised baselines. Hyperparameter sweeps over network width and regularization strength reveal clear performance optima, as visualized by the development-set F1-score heatmap in Fig. 5.

Comparing pixel-based and feature-based MLPs shows that engineered features consistently yield stronger performance, suggesting that domain-informed representations capture discriminative information that is difficult to learn from raw pixels alone in a shallow model.

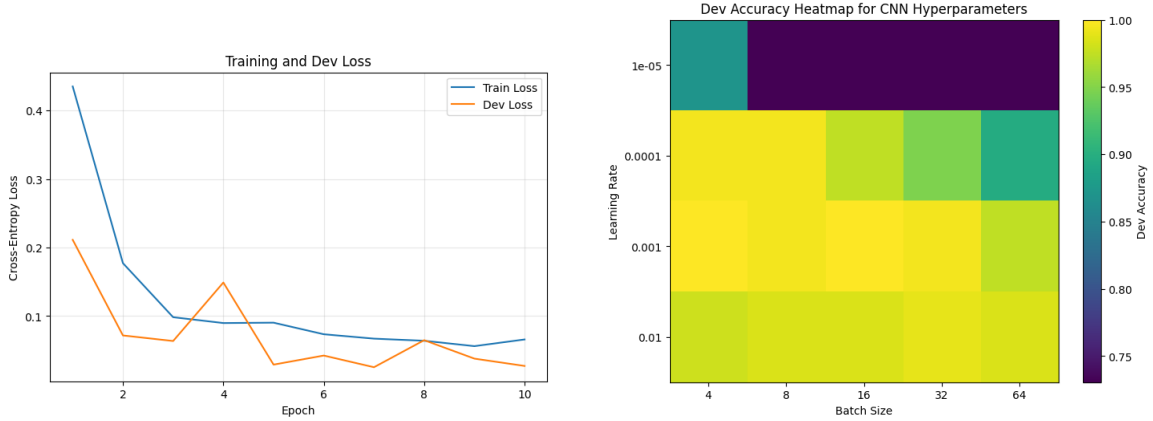


Figure 6: CNN training dynamics. Left: training and development loss curves. Right: accuracy as a function of learning rate and batch size on the development set.

4.5 CNN training dynamics and performance

The CNN trained end-to-end on raw images achieves the strongest overall performance. Figure 6 shows the training and development loss curves, indicating stable convergence without strong overfitting.

4.6 Comparative evaluation across methods

4.6.1 Performance comparison

Fig. 7 summarizes the performance of all five pipelines across accuracy, F1 score, and ROC AUC (metrics defined in Sec. 2.7). The overall pattern is consistent across metrics: supervised methods outperform unsupervised clustering baselines, and stronger representations yield consistently better results.

The weakest baseline is PCA+k-means on raw pixels, which performs close to chance-level in ROC AUC ($\text{Acc} \approx 0.33$, $F_1 \approx 0.35$, $\text{ROC AUC} \approx 0.46$), reflecting the poor class alignment already observed in the pixel PCA analysis. Switching to engineered features yields a large gain for unsupervised learning ($\text{Acc} \approx 0.75$, $F_1 \approx 0.68$, $\text{ROC AUC} \approx 0.82$), demonstrating that representation quality is critical when labels are not used during fitting.

When labels are available, both supervised baselines improve substantially. The pixel-PCA MLP reaches ($\text{Acc} \approx 0.88$, $F_1 \approx 0.77$, $\text{ROC AUC} \approx 0.83$), while the feature-PCA MLP improves further ($\text{Acc} \approx 0.95$, $F_1 \approx 0.91$, $\text{ROC AUC} \approx 0.93$). Finally, the CNN achieves perfect scores across all metrics ($\text{Acc} \approx 1.00$, $F_1 \approx 1.00$, $\text{ROC AUC} \approx 1.00$), indicating that end-to-end learning from raw images best captures the nonlinear and spatial structure relevant for distinguishing live and dead cells.

4.6.2 Error analysis

Table. 2 compares confusion matrices across all five pipelines and highlights how error patterns change with representation quality and supervision.

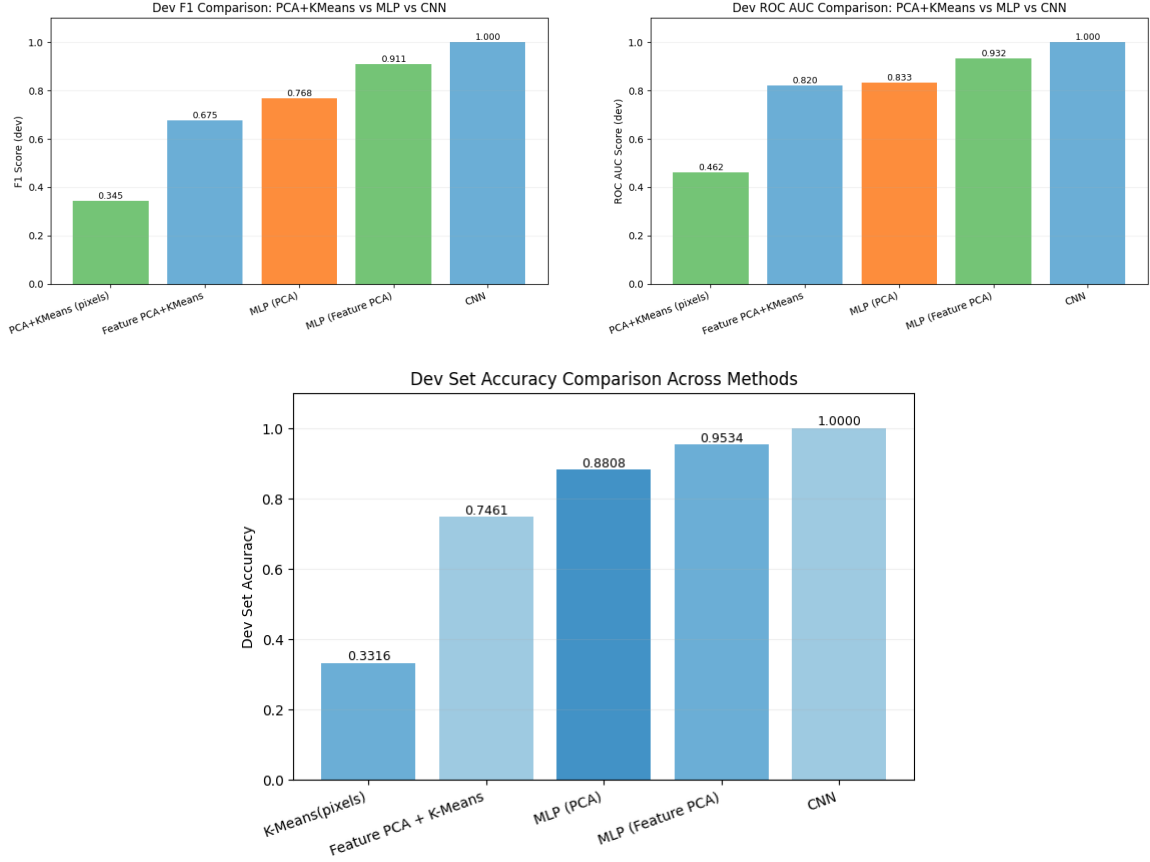


Figure 7: Comparison of model performance on the development set across multiple metrics.

The **pixel-based PCA+k-means** baseline exhibits a strong tendency to assign many live samples to the dead cluster, consistent with its weak ROC AUC and low F1 reported earlier. This pattern is typical when an unsupervised partition is only weakly aligned with the semantic classes: the clustering objective is driven by dominant variance directions rather than class structure, so one cluster can effectively “absorb” much of the majority class without producing a balanced separation. Moving to **engineered features** substantially improves the error profile: false negatives (dead predicted as live) become rare, while the remaining mistakes are dominated by **false positives** (live predicted as dead). This asymmetry suggests that the feature set captures cues associated with dead cells reliably, but that some live cells share similar feature signatures—an expected limitation when a compact, hand-crafted representation must summarize diverse cell appearances.

For the **supervised models**, the confusion matrices become increasingly diagonal. The pixel-PCA MLP reduces the large false-positive rate seen in unsupervised pixel clustering, indicating that label information helps the classifier focus on class-relevant directions even when the representation is imperfect. The feature-PCA MLP further decreases residual errors, consistent with combining a compact, informative representation with supervision. Finally, the CNN shows near-perfect separation on this split, with essentially no systematic bias toward one error type, indicating that the learned end-to-end features capture the discriminative structure that simpler representations and

Table 2: Confusion matrices on the development set. Rows are true labels (Live, Dead) and columns are predicted labels (Live, Dead). Diagonal entries indicate correct predictions.

Model	Confusion matrix
PCA+k-means (pixels)	$\begin{bmatrix} \mathbf{30} & 111 \\ 18 & \mathbf{34} \end{bmatrix}$
Feature PCA+k-means	$\begin{bmatrix} \mathbf{93} & 48 \\ 1 & \mathbf{51} \end{bmatrix}$
MLP (pixel-PCA)	$\begin{bmatrix} \mathbf{132} & 9 \\ 14 & \mathbf{38} \end{bmatrix}$
MLP (feature-PCA)	$\begin{bmatrix} \mathbf{138} & 3 \\ 6 & \mathbf{46} \end{bmatrix}$
CNN	$\begin{bmatrix} \mathbf{141} & 0 \\ 0 & \mathbf{52} \end{bmatrix}$

unsupervised objectives miss.

4.6.3 Computational cost comparison

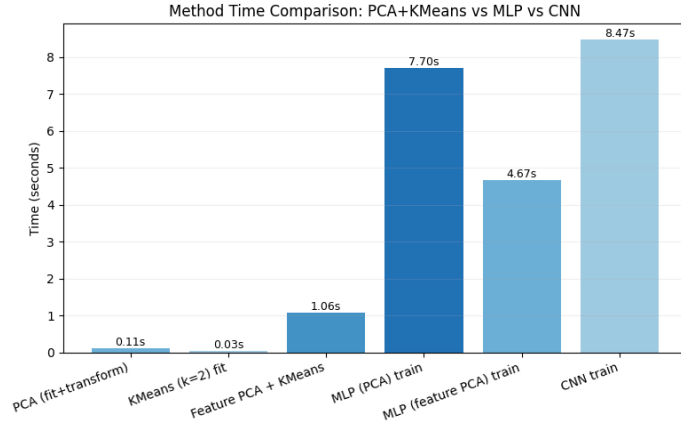


Figure 8: Comparison of computational cost across models.

Fig. 8 compares wall-clock runtimes for representative steps across the five pipelines (timing procedure described in Sec. 2.8). As expected, the unsupervised pixel-based baseline is computationally cheapest: PCA fitting and the subsequent k-means fit complete in a fraction of a second. Incorporating feature extraction increases runtime, since feature computation and preprocessing add overhead beyond the clustering step.

The supervised models require substantially more computation due to iterative optimization. The MLP training time depends on both the representation and convergence behavior (e.g., number of optimizer iterations), while the CNN has the highest cost because it learns features and a classifier

end-to-end over multiple epochs. These results should be interpreted as relative comparisons: absolute timings depend on hardware (CPU/GPU), system load, and software stack (Python version and library implementations), and may change with newer releases of the underlying libraries.

4.6.4 Final test set evaluation

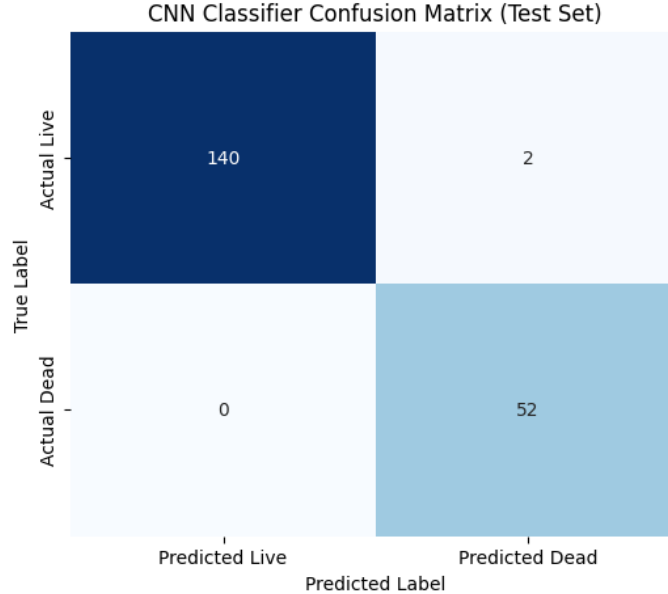


Figure 9: Comparison of computational cost across models.

Following the train–development–test protocol in Sec. 2.8, we evaluate the final selected model once on the held-out test set. Based on the development-set comparisons in Sec. 4.6.1, the CNN is selected as the final model. Fig. 9 shows its confusion matrix on the test set. The model generalizes strongly, with only a small number of misclassifications and no observed false negatives (dead predicted as live) in this evaluation. This result is consistent with the development-set performance reported earlier and supports the conclusion that the CNN captures discriminative spatial structure that is not fully accessible to the simpler feature-based or PCA-based pipelines.

5 Discussion

The experimental results highlight the strong dependence of both unsupervised and supervised learning performance on the chosen data representation. Across all settings, representation quality proves to be at least as important as model complexity. In particular, PCA applied to raw pixel data captures dominant sources of variance that are largely unrelated to the live/dead distinction, such as background intensity and illumination patterns. As a result, unsupervised clustering in pixel space performs poorly despite dimensionality reduction.

In contrast, engineered features substantially improve alignment between the data geometry and the biological classes. Even without access to labels, PCA followed by k-means on these features achieves

moderate F1 scores and ROC AUC, indicating that simple, biologically motivated descriptors can encode meaningful structure relevant to cell viability. This suggests that unsupervised methods are not inherently ineffective, but rather highly sensitive to whether the representation emphasizes task-relevant information.

Finally, several limitations should be noted. The dataset size is modest, and the near-perfect CNN performance raises the possibility that the task may be relatively easy given the controlled image conditions. We also observed intensity inconsistencies (including negative-like images), suggesting that more robust preprocessing and improved feature engineering could further stabilize the feature-based pipelines. Future work could evaluate robustness to distribution shifts, incorporate additional cell types or imaging conditions, and explore semi-supervised methods that bridge the gap between unsupervised clustering and fully supervised deep learning.

6 Conclusions

Based on the experiments and analyses presented in this project, we draw the following five main conclusions.

First, PCA of raw pixel representations are poorly suited for unsupervised clustering of live and dead cells. Even after dimensionality reduction with PCA, clustering in pixel space achieves low F1 scores, indicating that the dominant variance directions captured by PCA are not well aligned with the live/dead distinction. This underscores the limitations of applying unsupervised methods directly to high-dimensional pixel data without task-relevant representations.

Second, engineered features provide a substantially more informative and compact representation for unsupervised learning. When clustering is performed on PCA reduced feature vectors, performance improves markedly compared to raw pixels. PCA on engineered features captures most of the variance in a small number of components, and the resulting feature space aligns more closely with the live/dead distinction. While k-means clustering benefits significantly from this representation, its performance remains limited by the absence of label information.

Third, supervised learning methods consistently and substantially outperform unsupervised approaches across all representations. Multilayer perceptron (MLP) classifiers trained on PCA-reduced embeddings achieve strong performance, demonstrating that even relatively simple supervised models can effectively exploit low-dimensional representations when labels are available. In particular, MLPs trained on engineered feature embeddings outperform those trained on pixel-based PCA, confirming the value of meaningful feature design.

Fourth, feature-based MLP models offer a favorable balance between classification performance, interpretability, and computational efficiency. Compared to convolutional neural networks, these models are less expensive to train and easier to analyze, while still achieving high F1 scores. This makes them an attractive option in settings where computational resources or model transparency are important considerations.

Fifth, convolutional neural networks achieve the highest overall performance in this study. By directly exploiting the spatial structure of microscopy images, CNNs achieve near-perfect classification results and generalize well to the held-out test set. However, these performance gains come at the cost of increased computational complexity and reduced interpretability. Development-set-based model selection proves effective, as the selected CNN model maintains strong generalization performance on unseen data.

7 Deliverables

- Repository link: <https://github.com/Twaal/applied-data-analysis-and-ml/tree/main/Project3>
- Detailed instructions provided in the README.md file.

References

- [1] Recommendation itu-r bt.601: Studio encoding parameters of digital television for standard 4:3 and wide-screen 16:9 aspect ratios. Technical Report BT.601, International Telecommunication Union, Radiocommunication Sector (ITU-R), 1995.
- [2] Figshare. Trypan blue stained cells image dataset. https://figshare.com/articles/dataset/Trypan_Blue_stained_Cells_Image_Dataset/14818080, 2021. Dataset, DOI: 10.6084/m9.figshare.14818080.
- [3] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, Cambridge, MA, 2016.
- [4] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke, and Travis E. Oliphant. Array programming with NumPy, 2020.
- [5] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer, New York, NY, 2 edition, 2009.
- [6] Morten Hjorth-Jensen. *Computational Physics Lecture Notes 2015*. Department of Physics, University of Oslo, Norway, 2015.
- [7] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *International Conference on Learning Representations (ICLR)*, 2015.
- [8] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- [9] OpenAI. Chatgpt (gpt-5). <https://chat.openai.com/>, 2025. Large language model developed by OpenAI.
- [10] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [11] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, et al. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

- [12] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986.
- [13] Ryszard Tadeusiewicz and Przemysław Korohoda. *Komputerowa analiza i przetwarzanie obrazów*. Wydawnictwo Fundacji Postępu Telekomunikacji, Krakow, 1997.
- [14] René Vidal, Yi Ma, and S. S. Sastry. *Generalized Principal Component Analysis*. Interdisciplinary Applied Mathematics. Springer, New York, NY, 1 edition, 2016.